DOCUMENTAÇÃO TÉCNICA

RAG do BB Radar

Datathon Banco do Brasil

Arquitetura, implementação, validação e próximos passos do módulo de recuperação de informações sobre coberturas de seguro empresarial.

Projeto	BB Radar
Status	Primeira versão funcional
Tecnologias	Python, Pandas, Sentence Transformers e ChromaDB

Documento de apoio ao desenvolvimento e à apresentação técnica do projeto.

1. Objetivo do projeto

O BB Radar é uma solução em desenvolvimento para apoiar a análise e a recomendação de seguros empresariais. A proposta é utilizar informações das empresas e de seus estabelecimentos para identificar necessidades de proteção e apresentar recomendações justificadas.

O módulo RAG (Retrieval-Augmented Generation) foi iniciado para organizar informações sobre os planos de seguro empresarial e suas coberturas, permitindo recuperar dados relevantes a partir de perguntas em linguagem natural.

Escopo atual: preparação de documentos, geração de embeddings, armazenamento vetorial e recuperação de informações. A geração de respostas por um modelo de linguagem ainda não foi implementada.

2. O que é RAG?

RAG significa Retrieval-Augmented Generation, ou geração aumentada por recuperação. Em uma arquitetura completa, o sistema recupera documentos relevantes e os fornece a um modelo de linguagem, que usa esse contexto para elaborar uma resposta.

- O usuário faz uma pergunta.
- A pergunta é convertida em um embedding, uma representação numérica do texto.
- O sistema busca documentos com vetores semelhantes.
- Os documentos relevantes são recuperados.
- Em uma implementação completa, um modelo de linguagem pode usar esses documentos para formular uma resposta fundamentada.

No BB Radar, já foram implementadas as etapas de preparação, indexação e recuperação. Ainda não há geração de respostas com um modelo de linguagem.

3. Estrutura de arquivos

```
rag/
├── dados/
│   ├── catalogo_coberturas_fixas_planos.csv
│   └── empresas.csv
├── banco_vetorial/
├── criar_rag.py
├── consultar_rag.py
└── analisar_empresas.py
```

Componente	Responsabilidade
dados/	Armazena os arquivos CSV do projeto.
catalogo_coberturas_fixas_planos.csv	Relaciona planos às respectivas coberturas fixas.
empresas.csv	Contém informações empresariais para análises futuras.
criar_rag.py	Lê, limpa, transforma e indexa documentos no ChromaDB.
banco_vetorial/	Armazena persistentemente a coleção vetorial.
consultar_rag.py	Executa consultas semânticas e apresenta resultados.
analisar_empresas.py	Arquivo proposto para inspecionar a qualidade dos dados empresariais; execução ainda não confirmada.

4. Bibliotecas utilizadas

4.1 Pandas

O Pandas é utilizado para ler e manipular os CSVs. Os dados são carregados em um DataFrame, permitindo validar colunas, remover registros incompletos e eliminar duplicidades.

```
df = pd.read_csv(caminho_arquivo, encoding="utf-8-sig")
```

4.2 Sentence Transformers

A biblioteca converte textos em embeddings, representações numéricas que capturam aspectos do significado textual. O modelo utilizado é multilingue e suporta português.

```
modelo = SentenceTransformer("paraphrase-multilingual-MiniLM-L12-v2")

embeddings = modelo.encode(
    documentos,
    normalize_embeddings=True
).tolist()
```

Os embeddings não são respostas nem classificações de seguro: eles permitem comparar textos numericamente para recuperar documentos semelhantes.

4.3 ChromaDB

O ChromaDB armazena e consulta os documentos e seus embeddings. A configuração persistente permite reutilizar os dados entre execuções.

```
cliente_chroma = chromadb.PersistentClient(
    path="./banco_vetorial"
)

colecao = cliente_chroma.get_or_create_collection(
    name="catalogo_coberturas",
    metadata={"hnsw:space": "cosine"}
)
```

4.4 Ambiente virtual

As dependências foram instaladas em uma .venv para isolar as bibliotecas do projeto. A instalação inicial foi feita com:

```
python -m pip install pandas chromadb sentence-transformers
```

5. Preparação da base de conhecimento

A base inicial é o arquivo catalogo_coberturas_fixas_planos.csv, com as colunas plano e cobertura_fixa. Foram indexadas 21 associações entre planos e coberturas fixas, distribuídas entre ESSENCIAL, PRATICO e TOTAL.

5.1 Validação e limpeza

O script verifica a presença das colunas necessárias, remove linhas sem plano ou cobertura, remove espaços excedentes e elimina associações duplicadas.

```
colunas_necessarias = {"plano", "cobertura_fixa"}

if not colunas_necessarias.issubset(df.columns):
    raise ValueError(
        "O catálogo não possui todas as colunas necessárias."
    )

df = df.dropna(
    subset=["plano", "cobertura_fixa"]
).copy()
```

5.2 Transformação em documentos

Cada linha do CSV é transformada em uma frase. O texto foi aprimorado para ficar mais natural e manter também o identificador original da cobertura.

```
cobertura_legivel = cobertura_original.replace("_", " ").lower()

documento = (
    f"No plano de seguro empresarial {plano}, "
    f"uma das coberturas fixas é {cobertura_legivel}. "
    f"Identificador da cobertura: {cobertura_original}."
)
```

Essa transformação ajuda a representar o registro em texto, preservando a chave original para conferência.

6. Criação da base vetorial

6.1 Geração dos embeddings

Cada documento é convertido em um vetor pelo Sentence Transformers. A normalização dos vetores é usada para facilitar a comparação por similaridade cosseno.

6.2 Criação ou recuperação da coleção

O método get_or_create_collection recupera a coleção existente ou a cria quando ainda não existe. A métrica escolhida é a distância cosseno.

6.3 Armazenamento

```
colecao.upsert(
    ids=ids,
    documents=documentos,
    embeddings=embeddings,
    metadatas=metadados
)
```

Cada registro recebe um identificador e metadados com o plano e a cobertura. O método upsert insere ou atualiza os registros pelos seus IDs.

A execução bem-sucedida exibiu: Documentos indexados: 21 e Base vetorial criada em ./banco_vetorial.

7. Como funciona a consulta

O arquivo consultar_rag.py carrega o mesmo modelo, conecta-se à coleção persistente, recebe uma pergunta, converte-a em embedding e consulta os documentos semelhantes.

```
resultados = colecao.query(
    query_embeddings=embedding_pergunta,
    n_results=colecao.count()
)
```

A versão atual recupera todos os documentos do catálogo e ordena localmente as coberturas pela menor distância vetorial. Isso é aceitável para uma base pequena, mas deve ser revisto se o catálogo crescer.

Interpretação da distância

Na configuração utilizada, distâncias menores indicam maior proximidade entre os vetores. Essa proximidade não garante que o documento seja a resposta correta: a busca semântica não valida condições contratuais nem regras de elegibilidade.

8. Validação prática

Foi testada a pergunta: “Quais planos oferecem proteção contra danos elétricos?”

Na primeira consulta semântica, a busca também retornou responsabilidade civil antes de danos elétricos. Isso demonstrou que a similaridade textual isolada pode ordenar resultados que não respondem diretamente à pergunta.

Para validar a associação, foi aplicada uma filtragem exata pelo identificador DANOS_ELETRICOS. O resultado foi:

```
Planos que possuem a cobertura DANOS_ELETRICOS:
- ESSENCIAL
- PRATICO
```

O resultado confirma que esses dois planos estão associados à cobertura no catálogo usado no teste.

Depois, a consulta foi modificada para receber perguntas por input() e exibir as três coberturas mais próximas. Os testes com perguntas sobre danos elétricos, responsabilidade civil e o plano TOTAL foram relatados como bem-sucedidos. Ainda assim, a lógica final seleciona uma única cobertura e não interpreta plenamente todos os tipos de intenção.

9. Limitações atuais

9.1 Catálogo restrito

A base contém associações entre planos e coberturas fixas, mas não inclui necessariamente condições contratuais completas, limites de indenização, exclusões ou franquias. Esses detalhes não podem ser inferidos apenas pelo nome de uma cobertura.

9.2 Interpretação limitada

A versão atual seleciona a cobertura semanticamente mais próxima. Perguntas sobre todas as coberturas de um plano exigem uma lógica diferente de perguntas sobre qual plano inclui uma cobertura específica.

9.3 Sem geração de respostas

Ainda não foi integrado um modelo generativo. O módulo é, nesta fase, um sistema de recuperação semântica com armazenamento vetorial, que constitui uma etapa de uma arquitetura RAG.

9.4 Sem recomendação personalizada

Os dados empresariais ainda não foram integrados às regras de recomendação. O sistema não determina quais coberturas são mais adequadas para uma empresa específica.

9.5 Avaliação ainda inicial

Os testes básicos confirmam a execução do fluxo, mas ainda é necessário criar um conjunto de perguntas com respostas esperadas e medir se os documentos corretos aparecem entre os resultados.

10. Próximas etapas

- 1. Validar os dados empresariais. Inspecionar empresas.csv, tipos de dados, valores ausentes, duplicidades e consistência dos identificadores.
- 2. Integrar os estabelecimentos. Relacionar empresas e imóveis pelas chaves corretas, considerando que uma empresa pode possuir mais de um estabelecimento.
- 3. Estruturar o catálogo completo. Incorporar informações oficiais disponíveis sobre coberturas, condições e limites, sem inventar dados ausentes.

- 4. Implementar regras explicáveis. Criar regras documentadas de elegibilidade e recomendação com base nas características da empresa e do estabelecimento.
- 5. Integrar as fontes ao RAG. Recuperar evidências que ajudem a explicar as recomendações.
- 6. Gerar respostas fundamentadas. Se necessário, integrar um modelo de linguagem que utilize os documentos recuperados e sinalize informações que precisam de confirmação.
- 7. Preencher os templates de entrega. Gerar elegibilidade, plano recomendado, coberturas, justificativas, confiança e perguntas de confirmação nos formatos exigidos.
- 8. Avaliar o sistema. Testar recuperação, consistência das regras e qualidade das justificativas.

11. Conclusão

A primeira versão estabeleceu uma infraestrutura funcional para recuperação semântica de informações sobre coberturas de seguro empresarial. O catálogo foi carregado e validado, os registros foram transformados em documentos textuais, os embeddings foram gerados e 21 documentos foram indexados no ChromaDB.

A principal lição técnica é que a recuperação semântica deve ser combinada com consultas estruturadas e regras de validação quando é necessário confirmar fatos objetivos, como a associação entre planos e coberturas.

Para o BB Radar, esta base representa o início de uma arquitetura mais ampla, que deverá integrar dados empresariais, estabelecimentos, catálogo de seguros e regras explicáveis. O objetivo é produzir recomendações fundamentadas nas fontes disponíveis, com justificativas rastreáveis e sem apresentar suposições como fatos contratuais.